LedgerForge — Python Developer Guide

WorkforceOS · Section 1 · Core Payroll, Accounting & Employee Data Engine

One complete working pack for Tasks 1 to 20: what you receive, what you build, how every handoff works, and the exact answer to every common question.

---

1. Start here

This guide, together with **workbook 01 (01_LedgerForge_Python_Master_Dataset.xlsx)** and your original task brief, is everything you need for all 20 tasks.

Every task below has four short parts:

- **Inputs you already have**: the files, sheets and rules that task needs.
- **Decisions already made**: the choices that are normally argued about, settled for you.
- **Questions answered**: the doubts people usually have at that step.
- **Proof to submit**: what to attach to your pull request when the task is done.

If something is not clear

Use this order. It always ends with you moving forward.

1. Look in the task block in this guide.
2. Look in the **README** and **data_dictionary** sheets of workbook 01.
3. Pick the simplest sensible option. Write one line in the **Assumptions** section of your README (what you assumed and why). Continue.

An assumption written in your README is accepted as it is. You can always continue working.

How your work is submitted

- Everything lives in **your own GitHub repository**: code, documents, reports.
- One pull request per task. The pull-request link is the “GitHub link” that your reports ask for.
- Phase reviews and Reports 1, 2 and 3 are markdown files in the `reports/` folder (templates are in Section 10).

- Give your repository link and report links through the same submission channel your programme already uses for your work (for example GrowMar Connect).

---

2. Your starter kit

Item	What it is
Workbook 01	30 sheets: employees, salaries, attendance, leave, payroll answer key, accounting, invoices, expenses, tax and statutory values. Fake data only. Start with the README sheet, then data_dictionary.
This guide	Contracts, rules, decisions and proof lists for all 20 tasks.
Software	Python 3.12, Docker Desktop (Compose v2), Git, VS Code, curl or Postman, DBeaver (optional database viewer). All free.
Libraries (pin the versions)	FastAPI, Uvicorn, SQLAlchemy 2, Alembic, psycopg, Pydantic and pydantic-settings, PyJWT, bcrypt, pandas, openpyxl, Faker, WeasyPrint, Jinja2, XlsxWriter, Celery, redis-py, pytest, pytest-cov, httpx.

The company in the data is **Zenith Nova Technologies Pvt Ltd** (fictional): 745 employees ever employed, 441 active, 11 departments, 58 roles, data from January 2020 to **30 September 2026**, with detailed attendance, payroll and accounting from 1 April 2025.

---

3. Dependency register (what you need, and where it is)

What you need	Where it is	Needed in
Employee, payroll, accounting and tax data	Workbook 01	Tasks 2 to 20
The API contract (list of endpoints)	Section 6 of this guide	Tasks 1, 3, 14

What you need	Where it is	Needed in
Login, token and role rules	Section 5	Tasks 5 onwards
Pay rules, tax values, statutory values	Task 8 below and workbook sheets tax_slabs, tax_parameters, statutory_rates	Tasks 8, 17, 18
Accounting rules and chart of accounts	Tasks 10 to 12 below and sheet chart_of_accounts	Tasks 10 to 13
Correct answers to test against	Section 7 (key numbers) and the payroll_runs, payroll_items, journal sheets	Tasks 8, 12, 13
Deployment readiness checklist	Section 9	Tasks 16, 20
Hosting platform for the live link	Announced separately when final. Section 9 shows how to be fully ready before that.	Tasks 16, 20
Report and review templates	Section 10	All phase reviews and reports

4. Handoff register (what you publish, and how)

How a handoff works: you publish the item in your own repository at the path shown, link it from your README under a heading **Handoff package**, and mention it in your phase review. Other teams read it from your repository. There is nothing to email, upload or send anywhere else.

Item	When	Publish at	Format
Draft API contract	Day 2 (Task 1)	`docs/api-contract.md`	Copy of Section 6 plus any endpoint you add
Sample employee data (10 records)	Day 2 (Task 1)	`scripts/sample_employees.json`	JSON array of 10 employees, field names as in the employees sheet

Item	When	Publish at	Format
Master dataset and seed script	End of Phase 1 (Task 4)	`data/seed/` and `scripts/seed.py`	One command: `python -m scripts.seed --reset`
Login rules and demo accounts	Task 5	`docs/auth.md`	Section 5 plus the 3 demo usernames you chose
Payroll and payslip APIs	Task 9	`docs/api-contract.md` updated	Endpoint list with example requests and responses
Ledger and reports	Task 13	`docs/api-contract.md` updated	Same
Frozen API contract	Task 14	`docs/openapi-v1.json` and `docs/api-guide.md`	OpenAPI file plus a short guide
Live backend link	Tasks 16 and 20	README and report	URL plus health-check URL
Exports and data dictionary	Task 19	`docs/data-dictionary.md`	Table of datasets and columns

Task 1 handoff, step by step (the Day-2 package)

1. Put the two files in the repository: `docs/api-contract.md` and `scripts/sample_employees.json`. If you already used other file names, keep them. Only the links matter.
2. In `README.md`, add a heading **Handoff package** with a link to each file.
3. Create a git tag `handoff-day2` and push it (`git tag handoff-day2 && git push origin handoff-day2`).
4. In your Day-2 submission, give the repository link and the two file links.

That is the complete handoff. No other platform or person is involved; other teams simply read your repository.

---

5. Shared standards

Ports and variables (use exactly these)

Service	Local port	Base path
Your backend	8000	`/api/v1`
PostgreSQL	5432	database `ledgerforge`
Redis (from Task 15)	6379	

Variable	Local default
`ENV`	`local` (`production` when deployed)
`DATABASE_URL`	`postgresql+psycopg://wos:wos@db:5432/ledgerforge` (host `db` = the Compose service name)
`REDIS_URL`	`redis://redis:6379/0`
`JWT_SECRET`	`zenith-nova-local-dev-secret-key-change-in-production` (local only; deployment uses its own value)
`JWT_ALGORITHM`	`HS256`
`ACCESS_TOKEN_MINUTES` / `REFRESH_TOKEN_DAYS`	`15` / `7`
`DEMO_TODAY`	`2026-09-30` (use this whenever a report needs “today”, so your numbers match the workbook)
`SEED_ON_START`	`false` locally, `true` allowed in deployment

Commit `.env.example` with these names. Never commit `.env`.

Login and tokens

- `POST /api/v1/auth/login` with `{"username": "<work_email>", "password": "<password>"}` returns `{"access_token", "refresh_token", "token_type": "bearer", "expires_in": 900}`.

- `POST /api/v1/auth/refresh` with `{"refresh_token"}` returns a new pair. `GET /api/v1/auth/me` returns user id, employee id, role and name.
- Token = JWT, HS256, signed with `JWT_SECRET`. Claims: `sub` (user id like U-0001), `employee_id`, `role` (ADMIN, HR or EMPLOYEE), `type` (`access` or `refresh`), `iat`, `exp`, `jti`. Other services verify the same token with the same secret and algorithm.
- Usernames are the `work_email` values in the **users** sheet. The seed script gives every user the password `Zenith#<employee_id>` (for example `Zenith#EMP-0001`), stored as a bcrypt hash. Choose one ADMIN, one HR and one EMPLOYEE for testing. These are fake-data passwords; never reuse them anywhere real.
- A wrong login returns 401 with the same message whether the username or the password was wrong.

Conventions for every endpoint

- **Errors:** `{"error": {"code": "VALIDATION_ERROR", "message": "...", "details": [{"field": "email", "issue": "invalid format"}], "request_id": "..."}}`. Codes: VALIDATION_ERROR 422, UNAUTHENTICATED 401, FORBIDDEN 403, NOT_FOUND 404, CONFLICT 409, INTERNAL_ERROR 500. No stack traces in responses.
- **Pagination:** `page` (from 1) and `size` (default 20, max 100). Response `{"items": [...], "total": 441, "page": 1, "size": 20, "pages": 23}`.
- **Filters:** plain query names (`department_id=D01`), `q` for name or e-mail search, `from` and `to` for dates, `sort=field` or `sort=-field`.
- **Money in JSON** is a string with 2 decimals (`"54788660.03"`). **Dates** are ISO.
- **Health:** `GET /api/v1/health`, no login, returns `{"status": "ok", "service": "ledgerforge", "version": "...", "checks": {"database": "ok", "redis": "ok", "worker": "ok"}}`.
- **Deleting an employee** sets `is_active = false`; history stays.
- **Money rule:** never use float for money. Python `Decimal` and PostgreSQL `NUMERIC(14,2)`; round half up to 2 decimals.
- **Ids** stay as text from the workbook (EMP-0001, D01, R001 ...).

Roles and permissions

Resource	ADMIN	HR	EMPLOYEE
Employees	all	all (cannot change roles or users)	read own record
Users and roles	all	none	none
Salary structures and revisions	all	read and write	read own
Attendance	all	all	read own; mark own check-in
Leave requests	all, may approve any	all, may approve any	create own; a manager approves requests of direct reports (`manager_id` match); nobody approves their own
Payroll runs and adjustments	create, view, **approve**	create, view (no approve)	none
Payslips	all	all	own only
Ledger, expenses, invoices, reports	all	none	none
Tax slabs	read and write	read	none
Compliance reports	all	all	none
Exports	all	people and compliance exports	none

Finance staff in the data have the EMPLOYEE role, so ledger, expense, invoice and report access is **ADMIN only**. Do not add a fourth role. “Manager” is worked out from `manager_id`.

---

6. API contract v1 (final enough to build on from Day 1)

Base path `/api/v1`. Every endpoint needs a login except `/health` and `/auth/login`. Response field names follow the **column names of the workbook sheets**.

Area	Endpoints
Auth	`POST /auth/login`, `POST /auth/refresh`, `GET /auth/me`
Employees	`GET /employees`, `POST /employees`, `GET /employees/me`, `GET /employees/{id}`, `PATCH /employees/{id}`, `DELETE /employees/{id}`, `GET /employees/{id}/team`, `GET /employees/{id}/assignments`
Organisation	`GET /departments`, `GET /departments/{id}`, `GET /departments/{id}/employees`, `GET /roles`
Users (admin)	`GET /users`, `POST /users`, `PATCH /users/{id}`
Salary	`GET /salary/components`, `GET /employees/{id}/salary`, `GET /employees/{id}/salary/history`, `POST /employees/{id}/salary/revisions`
Attendance	`GET /attendance`, `POST /attendance`, `GET /attendance/summary?employee=me&month=2026-09`
Leave	`GET /leave/types`, `GET /leave/balance?employee=me`, `GET /leave/requests`, `POST /leave/requests`, `POST /leave/requests/{id}/approve`, `.../reject`, `.../cancel`, `GET /holidays`
Payroll	`POST /payroll/calculate` (dry run, one employee and month), `POST /payroll/runs`, `GET /payroll/runs`, `GET /payroll/runs/{id}`, `GET /payroll/runs/{id}/items`, `POST /payroll/runs/{id}/approve`, `GET /payroll/adjustments`, `POST /payroll/adjustments`

Area	Endpoints
Payslips	`GET /payslips/{employee_id}/{month}` (JSON), `GET /payslips/{employee_id}/{month}/pdf`, `POST /payslips/batch` (ZIP of PDFs)
Accounting	`GET /accounting/accounts`, `GET /accounting/accounts/{code}/balance?as_of=`, `GET /accounting/journal-entries`, `POST /accounting/journal-entries`, `GET /accounting/journal-entries/{id}`, `POST /accounting/journal-entries/{id}/reverse`
Business	`GET/POST /clients`, `GET/POST /vendors`, `GET/POST /expenses`, `POST /expenses/{id}/pay`, `GET/POST /invoices`, `GET /invoices/{id}`, `POST /invoices/{id}/send`, `POST /invoices/{id}/payments`
Reports	`GET /reports/trial-balance?as_of=`, `GET /reports/profit-and-loss?from=&to=`, `GET /reports/balance-sheet?as_of=`, `GET /reports/payroll-ledger-reconciliation?month=`
Tax and compliance	`GET/POST /tax/slabs`, `PUT /tax/slabs/{id}`, `GET /tax/calculate?annual=`, `GET /compliance/pf?month=`, `GET /compliance/esi?month=`, `GET /compliance/tds?month=`
Exports	`GET /exports/{dataset}?format=csv|xlsx&updated_since=`
Jobs and health	`GET /jobs/{job_id}`, `GET /health`

Job pattern (from Task 15): a long operation returns `202` with `{"job_id": "...", "status_url": "/api/v1/jobs/..."}`. `GET /jobs/{id}` returns `{"status": "pending|running|success|failed", "progress": {"done": 40, "total": 100}, "result": ..., "error": ...}`.

---

7. Seed script behaviour and key numbers

- Copy workbook 01 into `data/seed/`. Read it with `pandas.read_excel` (or export each sheet to CSV once with `scripts/export_sheets.py`).
- `python -m scripts.seed --reset` fills an empty database in one command and is safe to run twice.
- **Phase 1 and 2 seed loads:** departments, roles, employees (two passes because of the manager self-link), users (hashed passwords per Section 5), employee_assignments, salary_components, salary_structures, salary_revisions, performance_reviews, holidays, leave_types, leave_requests, leave_balances, attendance, tax_slabs, tax_parameters, statutory_rates.
- **Phase 3 seed adds:** chart_of_accounts, clients, vendors, invoices, invoice_items, invoice_payments, expenses, and every journal entry **except** those whose `source_type` is `PAYROLL_RUN`. Payroll postings are created by **your payroll engine and auto-posting** (Tasks 8 and 12) using `scripts/run_history.py`, which runs the 18 months in order.
- Keep the sheets payroll_runs and payroll_items under `tests/fixtures/` as the answer key. Do not load them into live tables.

Key numbers (from workbook 01) to check yourself:

Check	Value
Employees / active	745 / 441
Assignment rows / attendance rows	1,606 / 150,155
Run PR-2025-04	386 employees, gross 52,740,110.43, net 46,035,111.16, employer cost 53,363,392.08
Run PR-2026-09	444 employees, gross 54,788,660.03, net 48,089,678.30, employer cost 55,519,621.87
FY2025-26 (Apr 2025 to Mar 2026)	revenue 790,160,535.00, expenses 655,004,422.46, profit 135,156,112.54
Apr to Sep 2026	revenue 470,769,517.00, expenses 381,242,041.96, profit 89,527,475.04
At 30 Sep 2026	bank 290,962,361.45, receivables 139,394,278.00

---

8. The 20 tasks

Phase 1 — Foundation

Task 1 — Project skeleton & early handoff package

- **Inputs you already have:** Sections 5 and 6, sheets employees and departments.
- **Decisions already made:** folders `app/{api,services,models,schemas,core}`, `migrations/`, `scripts/`, `tests/`, `docs/`. Compose services `api` (port 8000) and `db` (postgres:16). Health endpoint `/api/v1/health`. Settings with `pydantic-settings`. The Day-2 package is the contract in Section 6 plus 10 sample employees; publish it as in Section 4.
- **Questions answered:** *Whom do I send the Day-2 files to?* Nobody; publishing them in your repository is the handoff (Section 4). *Where does the database host point?* To `db`, not `localhost`.
- **Proof to submit:** `docker compose up` screenshot, `/docs` page, passing health test, `.env.example`, the two handoff files linked in the README, tag `handoff-day2`.

Task 2 — Employee, department & role data model

- **Inputs you already have:** sheets departments, roles, employees, employee_assignments.
- **Decisions already made:** use the sheet column names as column names; text primary keys; self-link `manager_id`; add `created_at`, `updated_at`, `is_active`. One current role per person, history in `employee_assignments`. Unique `work_email`; check `exit_date >= hire_date`; indexes on department_id, manager_id, full_name, employment_status.
- **Questions answered:** a Mermaid diagram or a hand-drawn picture saved in `docs/` is enough for the ER diagram.
- **Proof to submit:** diagram, a migration that applies and rolls back, constraint tests.

Task 3 — Employee & team APIs

- **Inputs you already have:** Section 5 conventions and roles, Section 6 endpoints.
- **Decisions already made:** list filters `department_id, role_id, employment_status, work_location, work_mode, manager_id, hire_date_from, hire_date_to, q`. A "team" is the direct reports (`manager_id`). `DELETE` is a soft delete. Business rules live

in `services/`, not in routes. Validate e-mail format, hire date not after `DEMO_TODAY`, name not empty.
- **Proof to submit:** tests for every endpoint including bad input.

Task 4 — Master dataset, seed pipeline & early exports

- **Inputs you already have:** workbook 01 (this **is** the master dataset), Faker, pandas.
- **Decisions already made:** behaviour in Section 7. Use Faker in `scripts/extra_fake.py` to add 50 more employees with `Faker.seed(2026)`, kept in a separate file. CSV exports (employees, salary_structures, attendance) via `scripts/export_csv.py`, one file per table in `exports/`. The Kaggle HR dataset is optional: if you use one, pick a public dataset with an open licence and note its name and licence in the README; keep it separate from the master data.
- **Questions answered:** after seeding, row counts must equal the sheet row counts.
- **Proof to submit:** seed log, row-count table, CSV files, README note on data sources.

Phase 2 — Access control, payroll engine, payslips

Task 5 — Authentication & role-based access

- **Inputs you already have:** Section 5 (token rules and role table), sheet users.
- **Decisions already made:** bcrypt for passwords, PyJWT for tokens, one `require_role()` dependency used on every endpoint. 401 means not logged in, 403 means logged in but not allowed. A manager is worked out from `manager_id`.
- **Questions answered:** the demo accounts and passwords are in Section 5. Publish them (usernames only) in `docs/auth.md`.
- **Proof to submit:** a permission test table (3 roles x every endpoint), all rows passing; `docs/auth.md`.

Task 6 — Salary structure model

- **Inputs you already have:** sheets salary_components, salary_structures, salary_revisions.
- **Decisions already made:** a revision takes effect on the **1st of a month** (joining pay on the join date). A new revision closes the previous row the day before. Overlapping rows are rejected with 409. A revision for a month whose payroll run already exists is

rejected with 409. Split of monthly fixed gross: BASIC 40%, HRA 50% of basic, CONV 1,600 (800 if monthly gross is below 20,000), MED 1,250 (0 below 20,000), SPECIAL is the balance.

- **Proof to submit:** “pay on date X” tests for join day, revision day and exit day.

Task 7 — Attendance & leave model

- **Inputs you already have:** sheets attendance, leave_types, leave_requests, leave_balances, holidays.
- **Decisions already made:** attendance statuses are PRESENT, WFH, HALF_DAY_UNPAID, PAID_LEAVE, UNPAID_LEAVE, ABSENT_LOP. A leave request moves pending → approved, rejected or cancelled. CL 12 and SL 12 per financial year (reset on 1 April). EL grows 1.25 days a month and up to 30 days carry forward. If the chosen balance is too small the request becomes **LOP** (unpaid). The approver is the requester’s manager, or HR or Admin; never the requester. Write `count_unpaid_days(employee, month)` = ABSENT_LOP + UNPAID_LEAVE + 0.5 x HALF_DAY_UNPAID.
- **Proof to submit:** tests for overlapping requests, too little balance, cancel after approval; monthly summary equal to the sheet for 3 sample employees.

Task 8 — Payroll calculation engine

- **Inputs you already have:** the answer key (sheets payroll_items and payroll_runs, saved in `tests/fixtures/`), tax_slabs, tax_parameters, statutory_rates.
- **Decisions already made — the exact rules (the same rules the answer key was made with):**
  1. Fixed components come from the salary structure valid on the first day of the month (the join date for a joiner).
  2. `days_employed` = days from the later of hire date and month start to the earlier of exit date and month end, inclusive. `lop_days` comes from Task 7. `payable_days = max(0, days_employed - lop_days)`.
  3. Each component earned = round-half-up(component x payable_days / calendar_days, 2). `earned_fixed_pay` is the sum of the five rounded components.
  4. Overtime: only roles with `overtime_eligible = Y` (the role at month end). Hours = min(50, total overtime hours in the month from attendance). `overtime_pay` = round(hours x (monthly_fixed_gross / 208) x 2, 2).

5. `variable_pay` and `other_deductions` (loan EMI) are **inputs**, kept in a `payroll_adjustments` table. Seed it from the rows of payroll_items where variable_pay or other_deductions is above zero.
6. `gross_pay` = earned fixed pay + overtime pay + variable pay.
7. PF (permanent staff only): employee share = employer share = round(0.12 x min(basic earned, 15,000), 2).
8. ESI (permanent staff with monthly fixed gross of 21,000 or less): employee = round(0.0075 x gross, 2), employer = round(0.0325 x gross, 2).
9. Professional tax = 200 if gross is 25,000 or more, else 0.
10. TDS = round(annual_tax(monthly_fixed_gross x 12) / 12 x payable_days / calendar_days, 2). If variable pay is above zero, add round(annual_tax(fixed annual + variable) - annual_tax(fixed annual), 2).
11. `annual_tax(a)`: taxable = max(0, a - 75,000). Apply the slab table step by step. If taxable is 12,00,000 or less the tax is 0; otherwise tax = min(slab tax, taxable - 12,00,000). Then multiply by 1.04.
12. Safety rule: if total deductions are more than 85% of gross, set loan recovery to 0 and TDS = max(0, round(0.85 x gross, 2) - (PF + ESI + professional tax)).
13. `net_pay` = gross - total deductions. `employer_cost` = gross + employer PF + employer ESI.

- **Questions answered:** a run is **idempotent**: key `payroll:YYYY-MM:ZNT`; asking again returns the existing run. For the answer-key test, each row must be within 0.01 and each run total within 1.00.
- **Proof to submit:** 15 or more unit tests; an automated test that reproduces runs **PR-2025-04, PR-2025-10 and PR-2026-03**, then all 18 months.

Task 9 — Payslip generation

- **Inputs you already have:** sheets payroll_items and employees, WeasyPrint, Jinja2.
- **Decisions already made:** header = “Zenith Nova Technologies Pvt Ltd”, employee name, id, department, role, location, hire date, period, payable days, LOP days. Body = earnings (Basic, HRA, Conveyance, Medical, Special, Overtime, Variable), deductions (PF, ESI, Professional Tax, TDS, Other), gross, total deductions and **net pay**, a note on employer contributions, and the footer “Computer generated. Sample values for training.” JSON uses the same field names as payroll_items. A batch is one ZIP of PDFs named `<employee_id>_<YYYY-MM>.pdf`. Employees can fetch only their own payslip.

- **Proof to submit:** one PDF and one JSON showing identical numbers; a batch of 100 or more payslips. Update `docs/api-contract.md` with payroll and payslip examples.

Phase 2 review

Write `reports/phase-2-review.md` and **Report 1** (templates in Section 10).

Phase 3 — Accounting & ledger

Task 10 — Double-entry ledger model

- **Inputs you already have:** sheets chart_of_accounts (47 accounts), journal_entries, journal_lines.
- **Decisions already made:** an entry is accepted only if debits equal credits to the paisa. A line has either a debit or a credit. All lines of an entry are saved in **one database transaction**. Posted entries cannot be edited; a mistake is fixed with `POST .../reverse`, which creates an opposite entry. The cost centre on a line is a department id.
- **Proof to submit:** tests for balance, refusal of unbalanced entries, rollback, reversal.

Task 11 — Expenses & invoices

- **Inputs you already have:** sheets clients, vendors, invoices, invoice_items, invoice_payments, expenses.
- **Decisions already made:** invoice numbers look like `INV/2025-26/0001`, with a fresh sequence each financial year. GST is 18% for DOMESTIC clients and 0 for EXPORT clients. Postings:
  - **Invoice:** Dr 1100 (total), Cr revenue account (subtotal), Cr 2200 (GST).
  - **Receipt:** Dr 1001, Cr 1100.
  - **Expense bill:** Dr expense account (cost centre = department), Dr 1200 (GST), Cr 2001.
  - **Vendor payment:** Dr 2001, Cr 1001. Statuses go DRAFT → SENT → PAID, PARTIALLY_PAID or OVERDUE. DRAFT and CANCELLED post nothing. Each document stores the id of its ledger entry.
- **Proof to submit:** tests that each action posts a balanced entry; open invoice balances equal ledger account 1100.

Task 12 — Payroll-to-ledger auto-posting

- **Inputs you already have:** your payroll engine, the ledger, the key numbers in Section 7.
- **Decisions already made — exact postings.** On the **last day of the pay month**, one entry per run:
  - **Debits** (one line per department as cost centre; skip zero lines): 5001 earned fixed pay, 5002 overtime, 5003 variable pay, 5004 employer PF, 5005 employer ESI.
  - **Credits:** 2100 net pay, 2110 PF payable (employee + employer), 2120 ESI payable (employee + employer), 2130 TDS, 2140 professional tax, 1300 loan recoveries.
  - **Salary payment** on the first weekday of the next month: Dr 2100, Cr 1001.
  - **Remittances** (move to the next weekday if needed): TDS on the 7th, PF and ESI on the 15th, professional tax on the 20th of the next month, each Dr the liability, Cr 1001. The payroll run and its posting happen in **one transaction**; a failure rolls both back. Reversing a run posts opposite entries. Record who ran and who approved it.
- **Proof to submit:** `GET /reports/payroll-ledger-reconciliation?month=` shows difference 0.00 for all 18 months; audit rows for every step.

Task 13 — Financial reports engine

- **Inputs you already have:** the ledger and the key numbers in Section 7.
- **Decisions already made:** the trial balance lists every account with total debits and credits. Profit and loss = income minus expenses for a date range. The balance sheet as of a date has assets equal to liabilities plus equity, where equity includes the current-period profit. Contra account 1590 shows as a negative inside assets.
- **Questions answered:** after `scripts/run_history.py`, profit and loss for 2025-04-01 to 2026-03-31 must give revenue 790,160,535.00, expenses 655,004,422.46 and profit 135,156,112.54.
- **Proof to submit:** the three reports and the reconciliation test. Update `docs/api-contract.md`.

Phase 3 review

Write `reports/phase-3-review.md`.

Phase 4 — API contract, background jobs, first deployment

Task 14 — API versioning & documentation

- **Inputs you already have:** Section 6 and everything you built in Tasks 1 to 13.
- **Decisions already made:** every route is under `/api/v1/`. Every endpoint has a summary, description, tags, request and response examples, and its error responses. Save the OpenAPI file as `docs/openapi-v1.json`; a test compares the running API with that file. **Freeze rule:** from now on only additive, non-breaking changes (adding an optional field is fine; renaming or removing is not). Write `docs/api-guide.md`: how to log in, a first call, pagination, errors, roles.
- **Proof to submit:** the saved contract, a passing contract test, the guide.

Task 15 — Background jobs with a task queue

- **Inputs you already have:** the job pattern in Section 6, Redis settings in Section 5.
- **Decisions already made:** Celery with Redis as broker and result store; Compose services `redis` and `worker`. Tasks: `run_payroll(month)`, `generate_payslips_batch(month)`, `build_report(name, params)`. Retries: 3 times, waiting 10, 30 and 90 seconds. Responses follow the job pattern. A failed payroll run must leave no half-posted data.
- **Proof to submit:** start a run, show progress, force a failure, show that nothing was posted twice.

Task 16 — Mandatory deployment: backend

- **Inputs you already have:** the readiness checklist in Section 9.
- **Decisions already made:** the hosting platform is shared with you separately. Finish everything in the readiness checklist now; on the day the platform details arrive, follow the “Platform day” steps in Section 9.
- **Questions answered:** *Do I wait for the platform?* No. Your task is complete when the readiness checklist passes locally in production mode; the live link is added to your report as soon as you deploy.
- **Proof to submit:** local production-mode run, smoke-test output, and later the live link and health-check URL. Submit **Report 2** after Phase 4.

Phase 4 review

Write `reports/phase-4-review.md`.

Phase 5 — Tax, compliance, export, final release

Task 17 — Configurable tax slab module

- **Inputs you already have:** sheets tax_slabs and tax_parameters (sample values).
- **Decisions already made:** slabs and parameters live in database tables loaded from those sheets, per financial year with an `effective_from` date. Only **Admin** can edit. Payroll reads them from the database. A change affects pay months on or after `effective_from`; runs already made never change.
- **Proof to submit:** with the seeded slabs, payroll still matches the key numbers; after you change a slab, the next run changes while old runs stay the same.

Task 18 — Compliance & statutory reports

- **Inputs you already have:** payroll data and statutory_rates.
- **Decisions already made (columns):**
  - **PF:** employee_id, name, basic earned, PF wages (the smaller of basic and 15,000), employee share, employer share.
  - **ESI:** employee_id, name, gross wages, employee share, employer share (only ESI-applicable employees).
  - **TDS:** employee_id, name, annual fixed gross, annual tax, TDS deducted this month, year-to-date TDS. Output as JSON, CSV and Excel with a totals row. Totals must equal the payroll totals and the ledger payable lines. Label every file “sample values”.
- **Proof to submit:** a reconciliation test for three months.

Task 19 — Complete export & integration layer

- **Inputs you already have:** your tables (dataset names = sheet names of workbook 01).
- **Decisions already made:** every table has `updated_at`. `GET /exports/{dataset}?format=csv|xlsx&updated_since=ISO` streams CSV; Excel above 50,000 rows runs as a background job. The response header `X-Export-Watermark` carries the newest

`updated_at`; the next call sends it back as `updated_since`. Soft-deleted and reversed rows are included with their flag. Publish `docs/data-dictionary.md`.

- **Proof to submit:** an incremental-export test with no gaps and no duplicates; a large export that stays within memory.

Task 20 — Final integration & consolidated deployment

- **Inputs you already have:** everything above, plus Section 9.
- **Decisions already made:** end-to-end tests for three stories: hire to pay; invoice to payment to trial balance; tax slab change to statutory report. Coverage of at least 80% on `services/`. Regenerate the contract, run the contract test, tag `v1.0.0`. "Together with the other domains" means your service runs on the same platform using the ports and variables in Section 5; no code needs to be merged. Then write **Report 3**.
- **Final challenge (choose one and do it alone):** multi-company support; a payroll simulation endpoint ("what if everyone gets a 5% raise", no data changed); or a year-end tax statement PDF per employee.
- **Proof to submit:** test report, tag, live link, Report 3, a short description of your final challenge.

---

9. Deployment: be fully ready now, finish when the platform is shared

The hosting platform is announced separately when it is final. The name written in the brief is only a placeholder; ignore it until you receive the final platform. **No task is blocked** while you wait.

Readiness checklist (finish this during Tasks 14 to 16)

#	Item	How to know it is done
1	`docker-compose.prod.yml` with api, worker, db, redis	`docker compose -f docker-compose.prod.yml up` works on a clean machine
2	Container runs as non-root, debug off	`ENV=production` set
3	Every setting comes from an environment variable	README table lists name, purpose and example

#	Item	How to know it is done
4	Migrations run at start (`alembic upgrade head`)	Fresh database becomes ready by itself
5	Seed on first start when `SEED_ON_START=true`	Database filled in one go; second start does nothing
6	Health endpoint reports database, Redis and worker	`/api/v1/health` shows all “ok”
7	Logs go to the console (one line per request)	No log files needed
8	CORS limited by variable `CORS_ORIGINS`	No wildcard in production
9	No secrets in the repository; `JWT_SECRET` is at least 32 random characters in deployment	Secret scan of the repo is clean
10	Smoke script `scripts/smoke.sh`	Logs in, creates an employee, runs payroll, downloads a payslip, opens a report
11	README section “Deploy”	Steps below, filled with your own commands

Platform day (when the platform details are shared)

1. Create the service or project on the platform.
2. Create PostgreSQL and Redis (or attach the managed ones).
3. Set the variables from your README table: `ENV=production` , a new random `JWT_SECRET` , the platform’s `DATABASE_URL` and `REDIS_URL` , `CORS_ORIGINS` , and `SEED_ON_START=true` for the first deploy only.
4. Deploy your image or compose file.
5. Open the health URL and check every item is “ok”.
6. Run `scripts/smoke.sh` against the live URL.
7. Set `SEED_ON_START=false` , redeploy once, and write the link and a screenshot in your report.

---

10. Reports, self-review and templates

Phase review (end of every phase, 10 lines at most)

`reports/phase-N-review.md`:

- **Done:** tasks finished, each with its pull-request link.
- **In progress:** what is half done.
- **Blocked:** anything stuck, with the assumption you made to keep going.
- **Next phase plan:** two lines.

Report 1 (end of Phase 2), Report 2 (end of Phase 4, with the deployment link), Report 3 (end of Phase 5)

Section	What to write
Tasks done vs planned	Table: task, planned, done (yes or no), pull-request link
What I learned	5 plain lines
What this unblocks	Which documents or services other teams can now read, with links
Blockers	None, or the problem and the assumption you used
Demo	Screenshot or a 2-minute screen recording link
Deployment link	Reports 2 and 3 only (or “readiness complete, platform pending”)

Self-review before you merge any pull request

- [ ] All tests pass (`pytest`) and the completion points in the task block are met.
- [ ] The app starts from a fresh clone with one command.
- [ ] README and `docs/` updated; Assumptions section current.
- [ ] No secrets, no real personal data, no “test123” style fake values.
- [ ] Money uses Decimal; no float in any pay or ledger calculation.
- [ ] Evidence attached (screenshot, test output).

Professional quality checklist (before Phase 4 and Phase 5 deployments)

- [ ] Clear README with run steps.
- [ ] Complete and tidy API documentation.
- [ ] Realistic fake data.
- [ ] A one-page summary a non-technical person can follow (`docs/summary.md`).
- [ ] A clean final report.

Security self-check (do this before Report 2 and Report 3)

- [ ] Every endpoint has a role rule and a test.
- [ ] Passwords are hashed; tokens expire; refresh works.
- [ ] Employees cannot read other employees' payslips or salaries.
- [ ] Input is validated; errors never show stack traces.
- [ ] Audit rows exist for payroll runs, approvals and ledger postings.
- [ ] CORS is restricted; debug is off in production mode.
- [ ] Dependencies are pinned; no secrets in Git history.

Tool problems and the allowed fallback

Problem	Fix
WeasyPrint fails to build in Docker	Install `libpango-1.0-0 libpangoft2-1.0-0 libharfbuzz0b libffi-dev fonts-dejavu-core` in the Dockerfile
Celery will not run on Windows	Run the worker inside Docker
Reading the Excel workbook is slow	Convert each sheet to CSV once and seed from the CSV files
Seeding attendance is slow	Use bulk insert (`COPY` or `executemany` in batches of 5,000)
Docker Desktop will not start	Use a Linux VM or WSL2 and install Docker Engine there

Questions people ask most

- **Whom do I send the Day-2 files to?** Nobody. Publish them in your repository as shown in Section 4.
- **Where do I submit my work?** Repository link, pull-request links and report files go through the same submission channel your programme already uses.
- **Which platform do I deploy to?** It is shared separately. Be ready as in Section 9.
- **Do I have to use the Kaggle dataset?** No. Workbook 01 is the master dataset.
- **Which passwords do the demo users have?** `Zenith#<employee_id>` (Section 5).
- **My payroll differs from the answer key by one paisa.** Differences up to 0.01 per row are accepted. Check the rounding order in Task 8.
- **A rule is missing from this guide.** Use the simplest sensible option, write it under Assumptions in your README, and continue.
- **Do I need screens or a UI?** No. This domain is API only.